

Nodejs

```
// module

const { add, multiply } = require("../localModules");

const os = require("os");
const path = require("path");
const fs = require("fs");

// console.log(os.cpus());
// console.log(os.totalmem());
// console.log(os.freemem());

// console.log(os.hostname());
// console.log(os.homedir());

// console.log(os.platform());

// console.log(os.uptime());

// console.log(os.machine());

// console.log(os.userInfo());

console.log(path.basename("/src/index.js"));

console.log(path.extname("index.js"));

console.log(path.extname("index.pdf"));

console.log(path.join("src", "program", "index.js"));

console.log(path.dirname("/src/program/index.js"));

fs.writeFileSync("example.txt", "Hello express\n ");

fs.appendFileSync("example.txt", " second line");

const file = fs.readFileSync("example.txt");

console.log(file.toString());

// setTimeout(() => {

//     fs.unlinkSync("example.txt");
// }, 10000);
```

```
// global objects

console.log(__dirname);

console.log(__filename);

// timing function

setTimeout(() => {
  console.log("settimeout");
}, 3000);

const id = setInterval(() => {
  console.log("interval");
}, 3000);

setTimeout(() => {
  clearInterval(id);
}, 20000);

// bulit-in (nodejs has)
// local modules (developer created)
// third-party modules (npm )

// npm => node package manager

let data = 10;

let name = "Redmi";

data = "Fiit";
name = 50;

console.log(data, name);

let result = add(10, 40);
console.log(result);

multiply(20, 40, 5);
```

localmodules.js

```
// javascript => dynamically typed language

export function add( a, b){

    return a+b;

}

export function multiply(a, b, c){

    console.log( a*b*c);

}
```

Interview-Based Theory

1. Node.js Modules

Definition: Modules are reusable blocks of code that can be imported/exported. **Types:**

- Built-in modules → provided by Node.js (fs, path, os).
- Local modules → developer-created (require("./localModules")).
- Third-party modules → installed via npm (express, mongoose).

Syntax:

```
const fs = require("fs"); // built-in
```

```
const { add } = require("./localModules"); // local
```

2. File System (fs)

Definition: Provides methods to interact with files. **Key methods:**

- **fs.writeFileSync()** → create/write file.
- **fs.appendFileSync()** → append data.
- **fs.readFileSync()** → read file.
- **fs.unlinkSync()** → delete file.

3. Path Module (path)

Definition: Provides utilities for working with file and directory paths.

Key methods:

- **basename()** → returns filename.
- **extname()** → returns file extension.
- **join()** → joins paths.
- **dirname()** → returns directory name.

4. OS Module (os)

Definition: Provides system-related information. **Key methods:**

- **os.cpus()** → CPU info.
- **os.totalmem()** / **os.freemem()** → memory info.
- **os.hostname()** → system hostname.
- **os.platform()** → OS platform.

5. Global Objects

- **__dirname** → current directory path.
- **__filename** → current file path.

- `setTimeout()` → executes after delay.
- `setInterval()` → executes repeatedly.
- `clearInterval()` → stops interval.

Coding Round Questions

- Local module → Create a local module with `add()` and `multiply()` functions and import it.
- File system → Write a program to create, append, read, and delete a file.
- Path module → Demonstrate `basename`, `dirname`, `join`, and `dirname`.
- OS module → Print system memory and CPU details.
- Global objects → Show usage of `__dirname`, `__filename`, `setTimeout`, and `setInterval`.

Machine Coding Round Challenge

Problem: Log Manager System

Requirements:

- Create a local module `logger.js` with functions `logInfo()`, `logError()`.
- Use `fs` to write logs into `logs.txt`.
- Use `path` to manage file paths.
- Use `os` to log system info (`hostname`, `platform`).
- Use `setInterval()` to log a heartbeat message every 5 seconds.
- Stop logging after 20 seconds using `clearInterval()`.